DATA LAKE vs DATA WAREHOUSE vs DATA LAKEHOUSE V1

Three answers to the same question, asked in three decades under different constraints
what each solved · what actually changed · table formats · the catalog · cost models · choosing · migrating · **depth marked** junior → staff

1 · THESE ARE NOT THREE COMPETING PRODUCTS — THEY ARE THREE ERAS OF THE SAME PROBLEM

start here — each solved the previous one's failure, and the boundaries are now blurring in both directions

The question all three answer

Where do we put data so people can ask questions of it? The answer changed three times because the **economics changed** — not because the previous answer was foolish. Understanding what each was responding to explains their trade-offs better than any feature table.

The single technical shift underneath all of it: the separation of storage from compute. When storage and processing had to live on the same machine, you sized both together and paid for both permanently. Once cheap object storage decoupled them, you could **store everything and pay for compute only when querying** — and every architecture after that point is a consequence.

The sequence, and what forced each step

Warehouse 1990s–2000s	Storage was expensive, so you kept only what you had decided was valuable, in a schema fixed in advance. **Excellent for reporting; it discarded everything else**
Data lake 2010s	Object storage made keeping everything cheap. Store raw, decide the schema at read time. **Solved capture; many failed at retrieval**
Lakehouse 2020s	Add a **metadata layer over open files** so lake storage gains transactions, schema enforcement and warehouse-class performance. **The attempt to keep both properties**

Read that sequence as a response to failure, not as progress toward one correct answer. Each generation fixed the previous one's biggest weakness while introducing its own — and all three remain in production, frequently in the same organisation.

The historical lesson worth stating plainly: the data swamp. A large share of first-generation data lakes became unusable. Data went in and nothing came out — because there was **no catalog, no ownership, no schema enforcement and no quality gate.** Everything could be stored and nothing could be found or trusted. **The failure was governance, not technology**, and a lakehouse without governance fails exactly the same way.

What actually distinguishes them

Dimension	The real difference
Where structure is applied	Warehouse: on write. Lake: on read. Lakehouse: **on write for curated tables, on read for raw** — which is why it needs both disciplines
Who owns the storage format	Warehouse: historically the vendor. Lake and lakehouse: **you do**, in open files in your own object store
What guarantees writers get	Warehouse: full transactions. Lake: none. Lakehouse: **ACID from the table format**
Which engines can read it	Warehouse: its own. Lake and lakehouse: **any engine that understands the format**

That third row is the lakehouse's actual invention. Everything else was already possible on a data lake. Adding transactional guarantees to files on object storage is what made the lake usable for the workloads that previously required a warehouse.

STAFF **The honest 2026 position: the categories are converging, and from both directions. Snowflake** added managed Iceberg tables stored in your own cloud storage; **BigQuery** offers managed Iceberg tables in customer-owned buckets; **Databricks** added native Iceberg support. Warehouses now read and write open formats, and lakehouse engines have closed much of the performance gap. **The interesting question is no longer "which architecture" — it is which engine, and which catalog.**

Which means: be sceptical of comparison tables that are five years old, including several claims that circulate widely. Two that are now wrong: that warehouses cannot handle semi-structured data — they have had native JSON and variant types for years — and that they only scale up. Modern cloud warehouses are scale-out with separated storage and compute, which is the same architectural property the lakehouse is praised for.

How the decision usually presents in practice

- **It is rarely greenfield.** You have a warehouse and a growing lake, and the question is how they relate
- **The right answer is often both**, with one system of record for curated data and clear boundaries
- **Team capability matters more than the architecture diagram.** A lakehouse run by a team without data engineering depth becomes a swamp faster than a warehouse becomes expensive
- **Ask what you are optimising for** — cost, openness, BI concurrency, ML access or operational simplicity. They point to different answers, and only one of them can be first

2 · DATA LAKE

store everything, decide later

A **centralised store for raw data of any type** — structured, semi-structured and unstructured — held in its native format on cheap object storage. Structure is applied **when the data is read**, not when it is written.

What it does well

- **Ingest anything, immediately** — no schema negotiation before data can land
- **Very low storage cost**, so retention is an easy decision
- **Keeps the original**, so questions not yet asked remain answerable
- **Open access for any engine** — Spark, Python, SQL engines, ML frameworks all read the same files
- **Excellent for data science and ML**, which need raw detail rather than pre-aggregated summaries

What it does badly

- **No transactions.** A failed write leaves partial files, and a reader can see a half-written state
- **No schema enforcement** — a corrupted or changed producer is discovered downstream, by a person
- **Poor performance on complex SQL** without significant additional tooling
- **No update or delete semantics** — changing one row historically meant rewriting a partition
- **Governance is entirely external**, and therefore usually absent

The failure mode has a name: the data swamp. Without a catalog, ownership and quality gates, a lake accumulates data nobody can find, nobody trusts and nobody will delete. **The technology works exactly as designed** — permissive ingestion is the feature — and that is precisely the problem when nothing constrains it.

Where a plain lake is still exactly right: the **immutable raw landing layer** beneath any modern platform, ML feature engineering over full-fidelity history, long-term archival at the lowest storage cost, and exploration of data whose structure genuinely is not yet known.

3 · DATA WAREHOUSE

structured, governed, fast

A store of **cleaned, conformed, modelled data** optimised for analytical querying and business intelligence. Structure is enforced **on write**, so everything inside conforms to a declared schema.

What it does well

- **Complex SQL at high concurrency** — hundreds of simultaneous dashboard users is the workload it was built for
- **Full ACID transactions** and constraints, enforced by the engine
- **Governance built in** — access control, auditing and lineage as native features
- **Mature tooling** — every BI product connects, and every analyst already knows the interface
- **Operationally simple** — one system, managed, with far less to assemble and maintain

Two widely repeated criticisms are now out of date. "Cannot handle semi-structured data" — modern cloud warehouses have native JSON and variant types and query nested structures well. **"Scale-up only"** — they are scale-out with storage separated from compute, which is the same property the lakehouse is credited with. Both claims described the 2010s appliance era.

What remains genuinely limiting

- **Compute cost at sustained scale** — heavy continuous processing is often cheaper elsewhere
- **Format ownership**, historically. Data in proprietary storage is reachable only through that vendor's engine — **though managed Iceberg tables in customer-owned storage are actively dissolving this**
- **Weaker fit for ML** — training pipelines want direct file access to full-fidelity data, not a SQL interface
- **Unstructured content** — images, audio and documents do not belong here

Where a warehouse is still the best answer: BI and financial reporting where correctness and concurrency dominate, regulated environments needing provable governance, and any organisation whose team is small enough that **operational simplicity is worth more than openness.** That last case is more common than architecture discussions admit.

4 · DATA LAKEHOUSE

what it actually is

Stated precisely, so the term means something: a lakehouse is **open data files on object storage, plus a metadata layer that makes them behave like tables** — with transactions, schema enforcement and history — plus a catalog that governs access, plus engines that query it directly. **It is not a product. It is a stack, and each layer is a choice.**

```
query engines · BI · ML · SQL
        ↓
CATALOG — governance, discovery, access
        ↓
TABLE FORMAT — Iceberg | Delta | Hudi
  ACID · time travel · evolution
        ↓
FILE FORMAT — Parquet · ORC · Avro
        ↓
OBJECT STORAGE — S3 · ADLS · GCS
```

What it gets right

- **ACID on cheap storage** — the core achievement, and the thing a plain lake could not do
- **One copy, many engines** — no duplicating data into a warehouse to run BI on it
- **Time travel** — query a table as it was, which makes debugging and audit tractable
- **Schema and partition evolution** without rewriting history
- **Row-level updates and deletes**, including the ones privacy law requires
- **Storage you own**, in a format you can walk away with

What it costs, and this is routinely understated. It is more moving parts — format, catalog, engine and maintenance jobs are four separate decisions rather than one product. It needs **ongoing table maintenance** that nobody owns by default: compaction, snapshot expiry, orphan cleanup. And it needs a team with genuine data engineering depth. **A lakehouse is a capability commitment, not just an architecture choice.**

5 · THE THREE ARCHITECTURES SIDE BY SIDE — AND WHAT TECHNICALLY CHANGED BETWEEN THEM

where data flows, where structure is applied, and the two shifts that made each generation possible

DATA WAREHOUSE — structure first

Structured OLTP · ERP · CRM → **ETL** transform first → **WAREHOUSE** schema-on-write → **BI** dashboards · reports

Structure is decided before data is stored. Anything that does not fit is discarded.

- Data is cleaned, conformed and modelled *on the way in*. What lands is already trustworthy, and already committed to a shape.
- The engine owns the storage format, so it can optimise aggressively — indexes, statistics, clustering, caching, query planning.
- Transactions, constraints and access control are native, not assembled from parts.

Strength: unbeatable for BI concurrency, complex SQL and governed reporting. One system to operate.

Cost: you must know the questions in advance. Data judged not valuable is never captured, and that decision cannot be revisited.

The 1990s constraint that produced this shape: storage was expensive, and compute lived on the same machine as the data.

Modern cloud warehouses have since separated storage from compute and added native JSON and variant types — so two of the standard criticisms of this column no longer hold. What remains is format ownership and cost at sustained processing scale.

DATA LAKE — store first

Any source any format, any rate → **Load raw** no transformation → **OBJECT STORE** schema-on-read → **ML · Spark** exploration

Structure is applied when the data is read. Nothing is rejected on the way in.

- Cheap object storage removed the need to decide what is worth keeping. Keep everything, at full fidelity, indefinitely.
- Files are open and any engine can read them. No vendor stands between you and your own data.
- Ingestion is trivial — there is no schema to negotiate before data can land.

Strength: capture. Data science and ML get full-fidelity history rather than someone else's summary.

Cost: no transactions, no schema enforcement, no update semantics, and governance only if you build it separately.

The characteristic failure: the data swamp. Permissive ingestion with no catalog, no ownership and no quality gate produces a store where everything went in and nothing can be found or trusted.

Note that this was a governance failure rather than a technical one — which is why the same failure is available in a lakehouse.

DATA LAKEHOUSE — store first, with guarantees

Any source batch · stream · CDC → **TABLE FORMAT** ACID · time travel · schema evolution → **OPEN FILES** Parquet on object storage → **MANY ENGINES** SQL · BI · ML · streaming

Enforced on curated tables, permissive on raw. Both disciplines, in one store.

- The metadata layer is the whole idea: a transaction log and manifest over ordinary Parquet files turns a directory into a table.
- Writers get atomic commits and conflict detection. Readers never see a half-written state. Neither was possible on a plain lake.
- The catalog above it governs discovery and access, and determines which engines may participate.

Strength: one copy of the data, open format, warehouse-class guarantees, and BI plus ML from the same tables.

Cost: four decisions instead of one, and maintenance jobs — compaction, snapshot expiry, orphan cleanup — that nobody owns by default.

The enabling shift: object storage got cheap and consistent enough that a metadata layer could provide real transactional guarantees on top of it. That is a genuine advance, and it is narrower than the marketing suggests.

Governance is still yours to build. The swamp failure mode is fully available here if ownership and quality gates are skipped.

THE CONVERGENCE — why a 2020-era comparison table now misleads

Warehouses moved toward the lake. Snowflake added managed Iceberg tables stored in customer-owned cloud storage; BigQuery offers managed Iceberg tables in your own buckets; both read and write open formats. The proprietary-storage objection — for years the strongest argument against a warehouse — is materially weaker than it was.

Lakehouses moved toward the warehouse. Query engines closed much of the performance gap, catalogs added fine-grained governance, and the operational experience improved substantially. Databricks added native Iceberg support alongside Delta.

The practical consequence: the architecture label matters less than three concrete questions — which engine runs your queries, which catalog governs access, and whose storage account holds the bytes. Those determine cost, capability and how hard it would be to leave. Comparing "lake versus warehouse versus lakehouse" without answering them is comparing categories rather than options.

READING THE DIAGRAM — four properties that explain the rest of this sheet

1 · **Where structure is applied is the real axis.** On write means trustworthy but committed in advance; on read means flexible but unverified. The lakehouse does both at different layers, which is why it needs both sets of discipline rather than neither.

2 · **The metadata layer is the lakehouse's actual invention.** Transactions over open files on object storage is the specific advance. Everything else attributed to the lakehouse was already achievable on a data lake with enough effort.

3 · **Governance is orthogonal to all three.** The swamp was a failure of ownership, cataloguing and quality gates — not of the storage layer. No architecture on this sheet prevents it, and each will produce it if those are skipped.

4 · **The categories are converging, so decide at the layer below them:** table format, catalog, engine and storage ownership. Those four are the choices with consequences; the label is mostly how you describe the result afterwards.

6 · THE DETAILED COMPARISON — CORRECTED FOR WHERE THINGS ACTUALLY STAND

MID the reference table · STAFF the rows marked are where dated claims usually appear

Aspect	Data lake	Data warehouse	Data lakehouse
Primary purpose	Store raw data of any type at any scale	Curated, structured data for reporting and BI	Unified analytics across all data types
Data types	Structured, semi-structured, unstructured	Structured, **and semi-structured via native JSON and variant types**	Structured, semi-structured, unstructured
Where schema applies	On read	On write	On write for curated, on read for raw
Storage format	Native or raw — JSON, CSV, logs, Parquet	Engine-managed columnar; **increasingly open Iceberg tables in customer storage**	Open columnar (Parquet, ORC) plus a table format (Iceberg, Delta, Hudi)
Transactions	**None**	**Full ACID**, with constraints	**ACID** via the table format; constraint support varies
Performance profile	Good for scans and exploration; weak on complex SQL	**Strongest for BI concurrency** and complex joins	Strong and improving; depends heavily on layout and maintenance
Scalability	Massive, cheap object storage	**Scale-out with storage and compute separated** — the appliance-era "scale-up only" claim is obsolete	Massive, cloud-native
Governance	Entirely external — and therefore often absent	Built in and mature	Via the catalog — **capable, and it is a component you must choose and operate**
Cost shape	Very low storage; compute paid per job	Higher compute; **often lower total cost when usage is spiky, because it idles well**	Low storage; compute per engine; **plus ongoing maintenance jobs**
Engine access	Any engine that reads the files	Historically its own; **open formats now widen this**	Any engine supporting the table format and catalog
ML and data science	**Best** — direct file access to full history	Workable via connectors; not the natural path	**Best of both** — files for training, SQL for features
Operational burden	Low to run, high to make usable	**Lowest** — one managed system	**Highest** — format, catalog, engine and maintenance are separate concerns
Best suited for	Raw landing, archival, ML, unknown schema	BI, financial reporting, regulated analytics, small teams	Mixed BI and ML at scale, with open formats and a capable team

The row worth arguing about is operational burden. Comparison tables usually omit it, and it is decisive for smaller teams. A lakehouse asks you to choose and operate four things a warehouse provides as one. That is a fair trade at scale with a data platform team, and a poor one without.

7 · WHAT ACTUALLY ENABLED EACH ERA

the technical shifts underneath

Each generation became possible because something underneath changed. Naming those shifts explains the architectures better than listing their features.

1 · Cheap, durable object storage

- Storage became inexpensive enough that **"keep everything" stopped being a budget decision**
- Effectively unlimited, highly durable, and accessible over a network from anywhere
- **This alone created the data lake**

2 · Separation of storage and compute

- Compute no longer had to sit on the machine holding the data
- **Scale each independently**: pay for compute only while querying
- Several engines can read the same data at once without copying it
- **This is the deepest shift on the sheet** — and modern warehouses have it too, which is why the old comparison points blur

3 · Columnar file formats

- Parquet and ORC read only the columns needed, compress far better, and carry statistics that let engines skip whole blocks
- **They made querying files competitive** with a purpose-built engine

4 · The metadata layer

- A transaction log and manifests over ordinary files, giving **atomic commits, isolation and history**
- **This is the lakehouse's specific invention**, and the reason the term exists at all

Understanding these four makes the trade-offs predictable. Anything requiring transactional guarantees needs the metadata layer. Anything requiring high BI concurrency needs an engine that caches and plans well. Anything requiring low cost needs compute that idles cheaply. The architecture label follows from those requirements rather than preceding them.

8 · OPEN TABLE FORMATS — WHERE THEY GENUINELY DIFFER

a feature matrix of all ticks is not useful · STAFF the differences that survive scrutiny

Capability	Delta Lake	Apache Iceberg	Apache Hudi
ACID transactions	Yes	Yes	Yes
Schema evolution	Yes	Yes	Yes
Time travel	Yes	Yes	Yes
Partition evolution	**Via clustering**	**Native**	**Limited**
Hidden partitioning	**No**	**Yes**	**No**
Row-level updates	Yes	Yes	**Optimised for it**
Streaming ingest	**Strong**	**Good**	**Strongest**
Incremental pull	**Change feed**	**Incremental scan**	**Core design goal**
Metadata mechanism	Transaction log	Manifest tree	Timeline
Engine breadth	Broad, deepest on Spark	**Broadest and most vendor-neutral**	Narrower

The matrix you usually see gives all three a tick everywhere. That is roughly true at feature level and unhelpful for choosing. **The rows in bold are where they actually diverge**, and where the decision should be made.

What each is genuinely best at

Iceberg	**Vendor neutrality and the broadest engine support.** Partition evolution lets you change the physical layout **without rewriting history**, which directly de-risks one of the hard-to-reverse decisions. Hidden partitioning means queries need not know the scheme to benefit from it. **The default choice when openness is the priority**
Delta Lake	Mature, excellent tooling, and the deepest integration with Spark and Databricks. **The natural choice if that is your platform** — and Databricks now supports Iceberg natively as well, so this is less binding than it was
Hudi	Built around **upserts and incremental consumption**. The strongest fit for CDC-heavy ingestion where records update constantly and downstream consumers want only what changed

Partition evolution deserves the emphasis it gets here. Physical layout is one of the genuinely expensive things to change on a large table — rewriting petabytes is slow and costly. Being able to change the partitioning scheme going forward, without touching existing data, converts a structural decision into a reversible one.

The interoperability point. The **Iceberg REST catalog specification** has become the common interface, implemented across major warehouses and engines, and cross-engine read and write against the same tables is now real. That reduces the cost of choosing wrongly — but it also means **the format is no longer where lock-in lives.** See the next panel.

A practical note on picking. Most organisations should choose based on which engines they already run and which catalog they will use, not on the feature matrix. All three are capable; the integration cost of the odd one out is what you will actually feel.

9 · THE CATALOG

where lock-in lives now

The most under-discussed component on this sheet. If the table format decides how files behave, the **catalog decides which tables exist, who may read them, and which engines can participate.** Open files governed by a closed catalog are not as open as they appear.

What a catalog does

- **Table registry** — what exists, and where its current metadata pointer is
- **Atomic commits** — swapping that pointer is what makes a write atomic across engines
- **Access control** — increasingly the enforcement point, down to rows and columns
- **Discovery and documentation** — ownership, descriptions, business glossary
- **Lineage**, where it integrates with the engines

The options

Iceberg REST catalog	**The open specification** that has become the common interface. Implemented across major warehouses and engines — the thing to check for before anything else
Apache Polaris	An open-source REST catalog implementation, now a top-level Apache project
Unity Catalog	Databricks' governance layer, with an Iceberg REST endpoint for external engines
AWS Glue	The default on AWS; broad service integration
Hive Metastore	Long-standing and widely supported. **Legacy for new builds** — a REST catalog is the current answer
Nessie	Adds git-like branching and tagging over table state — genuinely useful for testing data changes in isolation

The questions to ask before committing. Does it implement the open REST specification? Can multiple engines **write**, not merely read? Is access control enforced consistently for every engine, or only the vendor's own? **And what happens to your metadata if you leave?** Open files with metadata you cannot export is a subtler lock-in than the one everyone was worried about.

10 · CHOOSING

a decision framework

If this is true	Lean toward
Workload is BI and reporting on structured data, with high concurrency	**Warehouse.** It is what the shape was built for, and it will be simpler and probably cheaper
Small team, no dedicated data platform engineers	**Warehouse.** Operational simplicity is worth more than openness when nobody can run the alternative
Heavy ML and data science on full-fidelity history	**Lakehouse**, or a lake beneath a warehouse
Mixed BI and ML, at scale, with a capable platform team	**Lakehouse.** One copy serving both is a real saving
Very large volumes with sustained processing cost	**Lakehouse.** Compute economics dominate at this scale
Avoiding vendor lock-in is a stated priority	**Lakehouse** with Iceberg and an open catalog — **and check the catalog, not just the format**
Unstructured content, or schema genuinely unknown	**Lake** as the landing layer, whatever sits above it
Strict regulatory governance with provable controls	**Warehouse**, or a lakehouse with a mature catalog and real ownership
You already run one and it mostly works	**Improve it.** Migration cost is routinely underestimated and rarely the highest-value work available

The most common correct answer is both. A lake or lakehouse as the landing and processing layer, a warehouse or a fast query engine serving BI. What matters is that **curated data has one system of record** and the boundary is explicit — not which logo sits in which box.

The question that settles most debates: who will operate this in two years? An architecture your team cannot run well is worse than a simpler one they can. This is the factor most often left out of the comparison and most often decisive afterwards.

11 · COST MODELS

how each actually bills you

"Lakes are cheap and warehouses are expensive" is too crude to be useful. Storage is cheap everywhere. **Compute dominates in all three**, and which is cheaper depends on your usage pattern rather than on the architecture.

	What you pay for
Lake	Object storage per GB, plus **request charges** — which small files inflate badly. Compute per job on whatever engine you run
Warehouse	Compute time or bytes scanned, plus managed storage. **Idles well** — auto-suspend means bursty usage costs little between bursts
Lakehouse	Cheap storage, compute per engine, **plus maintenance jobs** — compaction and cleanup are real recurring compute nobody budgets for

Where the money actually goes

- **Idle compute** — clusters and warehouses running with nothing querying. Usually the largest single saving available
- **Full scans** from partitioning or clustering that does not match query filters
- **Small files** — request overhead and wasted compute. A genuine cost, not just a performance annoyance
- **Duplicated data** — the same dataset in the lake and copied into the warehouse, processed twice
- **Forgotten pipelines** feeding dashboards nobody opens
- **Over-frequent refresh** — hourly rebuilds of weekly-reviewed data

The pattern that determines which is cheaper: spiky, interactive usage favours a warehouse, because it idles cheaply and you buy no baseline. **Sustained heavy processing favours a lakehouse**, because you control the compute and can use cheaper, interruptible capacity. Estimate against your actual query pattern before accepting either general claim.

Whichever you choose, tag everything from day one. Cost that cannot be attributed to a team or a dataset is cost nobody owns — and backfilling attribution across an existing estate is nearly impossible.

12 · MIGRATION PATHS

realistically

From → to	How it actually goes
Lake → lakehouse	**The easiest path.** The files are already there; you add a table format over them and register them in a catalog. Many tables convert in place without rewriting data. **Start here if you have a lake**
Warehouse → lakehouse	The hardest. Data must be extracted, transformations rewritten, and every downstream consumer repointed. **Do it incrementally, domain by domain**, never as a single cutover
Lake → warehouse	Straightforward for structured subsets. You are choosing to give up openness for simplicity, which is a legitimate trade
Adding a warehouse beside a lakehouse	Common and sensible — a fast serving layer for BI over curated tables. Increasingly the warehouse can read the same open tables directly

What makes migrations fail

- **Big-bang cutover.** Run both in parallel, reconcile outputs, and move consumers gradually
- **Not knowing who consumes what** — lineage before migration, or you will discover dependencies by breaking them
- **Rewriting transformations without reconciling** — **compare outputs row by row** before switching anything off
- **Underestimating the semantic work.** Moving data is the easy part; agreeing what each metric means is where the time goes
- **No rollback plan.** Keep the old system running until the new one has proved itself across a full reporting cycle

The uncomfortable question worth asking first: is migration the highest-value work available? A working warehouse with good governance beats a half-migrated lakehouse for years. **Migrate for a concrete constraint** — cost at scale, ML access, format ownership — not because the current architecture has an older name.

The low-risk first step, if you are unsure: land raw data immutably in object storage *alongside* whatever you run today. It is cheap, it is not a migration, and it preserves every future option — including the one where you never migrate.

13 · WHAT DOES NOT CHANGE — THE PARTS EVERY ARCHITECTURE STILL NEEDS

STAFF the disciplines that determine success, and that no architecture choice provides

The most useful thing to know when comparing these three: the hard parts are identical across all of them. Choosing an architecture settles roughly a quarter of the problem. The rest is the same work regardless, and it is where platforms actually succeed or fail.

Dimensional modelling still applies

- **Facts and dimensions remain the right shape** for BI serving — comprehensible, performant, and already understood by analysts
- **Slowly changing dimensions** are still a decision per attribute. Overwrite and lose history, or version and keep it
- **Conformed dimensions** — one definition of customer, product and date shared across marts — remain the difference between a coherent platform and a set of disconnected reports
- **"Just store it flat and query it" does not survive contact with real reporting requirements**

Data quality is not architectural

- Tests at every boundary — uniqueness, not-null, referential integrity, accepted values
- Monitoring for **freshness, volume, schema and distribution**
- An explicit decision per dataset: does a failed check **stop the pipeline or flag and continue?**
- A published SLA, so "the data looks wrong" becomes a measurement rather than an argument

Governance is orthogonal to storage

- **Every dataset needs a named owner.** A catalog without ownership is documentation, and it decays
- **Lineage** — where a number came from, and who breaks if this changes
- **Classification** of personal and regulated data, ideally automatic
- **Access control** granted to groups, at row and column level where needed
- **A deletion process** that reaches raw files, derived tables, exports and backups

This is the lesson of the data swamp, and it generalises. The lake did not fail because object storage was inadequate — it failed because nothing above it enforced ownership, discovery or quality. **The same failure is fully available in a lakehouse and in a warehouse**, and the architecture will not prevent it.

The engineering disciplines are constant

- **Idempotency** — orchestrators retry, so non-idempotent transforms duplicate data eventually
- **Immutable raw capture** — the only thing that makes reprocessing possible, in every architecture
- **Version control, testing and CI** for transformations. They are code
- **Backfill as a first-class, throttled operation**
- **Data contracts** with producers — the durable fix for upstream schema change
- **Cost attribution** from day one, because it cannot be added retrospectively

Which reframes the whole comparison. If two organisations pick different architectures and one does the above well, that one wins — regardless of which box they chose. **Architecture is necessary and nowhere near sufficient.**

14 · THE OPERATIONAL REALITY

what the diagrams omit

The small files problem — the most common lakehouse complaint, and it is absent from every architecture diagram. Frequent writes, streaming ingestion or per-partition micro-batches produce thousands of tiny files. Queries then open thousands of objects and **metadata and request overhead dominates the actual reading**. Performance degrades steadily with no change to data volume or query text.

Maintenance jobs a lakehouse needs, and a warehouse handles for you

Compaction	Rewrite many small files into fewer large ones. **The highest-value maintenance task**, and the one most often unscheduled
Clustering	Co-locate related rows so queries skip more data
Snapshot expiry	Remove old versions. **Trades time-travel depth against storage cost** — choose the retention rather than inheriting it
Orphan cleanup	Delete files left by failed writes that no metadata references
Statistics	Keep query planning accurate as data grows

This is the clearest concrete example of the operational trade. A managed warehouse does all of this invisibly. A lakehouse gives you control over it, which means **you now own it** — and a lakehouse without scheduled maintenance degrades quietly until someone investigates a slow dashboard and finds a table with millions of files.

Also frequently omitted

- **Concurrency limits** — table formats detect write conflicts, and heavy concurrent writers to one table will retry and fail
- **Time travel costs storage**, and retention must be set deliberately
- **Catalog availability is a dependency** — if it is down, tables cannot be resolved by any engine
- **Version compatibility** across engines, formats and catalogs is real coordination work

Schedule maintenance from the first table. It is unglamorous, it belongs to nobody by default, and its absence is the most common reason a lakehouse is judged slow.

15 · WHAT GOES WRONG — SYMPTOM, CAUSE AND FIX

the failures specific to each architecture, and the ones common to all three

Symptom	Cause	Fix
Data goes in, nothing comes out	**The data swamp.** No catalog, no ownership, no quality gate	Catalog with owners; classification; quality gates. **Not a storage problem**
Queries slower every month, same data	Small files accumulating; no compaction scheduled	Schedule compaction and clustering; monitor file counts per partition
Readers see partial data	Plain lake with no table format — a half-written directory is visible	Adopt a table format. This is precisely the problem it solves
Warehouse bill grew sharply	Warehouses not auto-suspending, or full scans from poor clustering	Auto-suspend aggressively; cluster on filter columns; attribute spend per team
Cannot delete a customer's data	Personal data spread across immutable files, copies and backups	Consolidate personal data; row-level deletes; crypto-shredding for backups

Symptom	Cause	Fix
Migration stalled halfway, two systems running	Big-bang plan, unknown consumers, unreconciled outputs	Lineage first; domain by domain; reconcile before switching anything off
Same data in lake and warehouse, processed twice	No agreed system of record for curated data	Pick one; let the other read it. **Open formats increasingly make this possible**
"Open format" but still cannot switch engines	The catalog is proprietary — the files are open, the metadata is not	Check REST spec support and metadata export **before** committing
Lakehouse slower than the warehouse it replaced	No maintenance, poor layout, or an engine not tuned for BI concurrency	Compaction and clustering; a serving engine suited to concurrency
Numbers disagree between systems	The same metric defined twice, in two places	A semantic layer — define once, serve everywhere
Cannot reprocess data from last year	Raw was transformed on ingestion, or not retained	**Land raw immutably.** There is no retrospective fix

What breaks first, in each

	First failure
Lake	**Trust.** Nobody knows what is in it or whether it is correct. Everything else follows from that
Warehouse	**Cost, then flexibility.** The bill grows, then a workload arrives that does not fit the model
Lakehouse	**Operational burden.** Maintenance nobody owns, four components to keep compatible
All three	**Governance and ownership.** By a wide margin the most common failure, and the one no architecture prevents

Note what is absent from that list: scale. All three handle large volumes competently. The failures are trust, cost, operations and ownership — none of which are solved by choosing a different storage architecture.

The questions worth asking of any of them

- **Who owns this dataset**, and who is called when it is wrong?
- **How do I find a table** without asking a person?
- **How do I fix data that was wrong last month?**
- **What does this dashboard cost to run?**
- **If we wanted to change engine, what would it take?**

16 · DECISION & READINESS CHECKLIST

before committing to an architecture

- ☐ **The actual constraint named** — cost, ML access, concurrency, openness
- ☐ **Current workload measured**, not assumed
- ☐ **Query pattern known** — spiky and interactive, or sustained
- ☐ **Team capability assessed honestly** against operational burden
- ☐ **"Who operates this in two years" answered**
- ☐ **Comparison claims checked for currency** — several common ones are dated
- ☐ **Raw data landed immutably**, whatever sits above it
- ☐ **Raw retention set** against reprocessing needs
- ☐ **Table format chosen on the rows where they differ**, not the ticks
- ☐ **Partition evolution need considered** — it de-risks layout decisions
- ☐ **Catalog chosen as deliberately as the format**
- ☐ **Catalog implements the open REST spec**; multi-engine *write* verified
- ☐ **Metadata export path confirmed** — could you leave?
- ☐ **Access control enforced consistently** across every engine
- ☐ **Compaction scheduled** and file counts monitored
- ☐ **Snapshot expiry and orphan cleanup running**
- ☐ **Time-travel retention set deliberately**, with its storage cost understood
- ☐ **Maintenance compute budgeted**, not discovered
- ☐ **Partitioning matches query filters**; no over-partitioning
- ☐ **Concurrent-writer conflicts considered** on hot tables
- ☐ **Catalog availability treated as a dependency**
- ☐ **Every dataset has a named owner**
- ☐ **Catalog populated automatically**, not by hand
- ☐ **Lineage captured**, column level where possible
- ☐ **Sensitive data classified** and access granted to groups
- ☐ **Deletion process designed and tested** end to end
- ☐ **Quality tests at every boundary**
- ☐ **Freshness, volume, schema and distribution monitored**
- ☐ **Stop-or-continue decided per dataset** on failed checks
- ☐ **Every transform idempotent**
- ☐ **Backfill a first-class, throttled operation**
- ☐ **Transformations version controlled and tested**
- ☐ **Dimensional model designed**; SCD strategy chosen per attribute
- ☐ **Conformed dimensions agreed** across domains
- ☐ **Metrics defined once** in a semantic layer
- ☐ **One system of record for curated data**; no duplicate processing
- ☐ **Everything tagged** for cost attribution from day one
- ☐ **Idle compute auto-suspended**; budgets and anomaly alerts set
- ☐ **Migration incremental with reconciliation**, never a cutover
- ☐ **Rollback plan held** through a full reporting cycle

The one that summarises the sheet: **the architecture label settles roughly a quarter of the problem.** Ownership, discovery, quality, idempotency, immutable raw capture and cost attribution are identical work in all three — and they are what actually separates a platform the business relies on from one it works around. Choose deliberately, then spend your effort on the parts the choice does not cover.